

Prog. Orientée Objet

Intro : POO inventée avant le C++

POO en C: struct Montype {
 Etat...
};

void Montype_init (Montype *, ...);

void Montype_update (Montype *, ...);

Montype var;

Montype_init (&var, ...);

Montype_update (&var, ...);

C++ originellement $\leftarrow$ C with classes $\rightarrow$

```
class MonType {
```

```
public:
```

```
    MonType();
```

```
    void Update();
```

```
private:
```

```
    MonEtat ...
```

```
};
```

Instance

```
MonType var;  
var.update(—);
```

Constructor

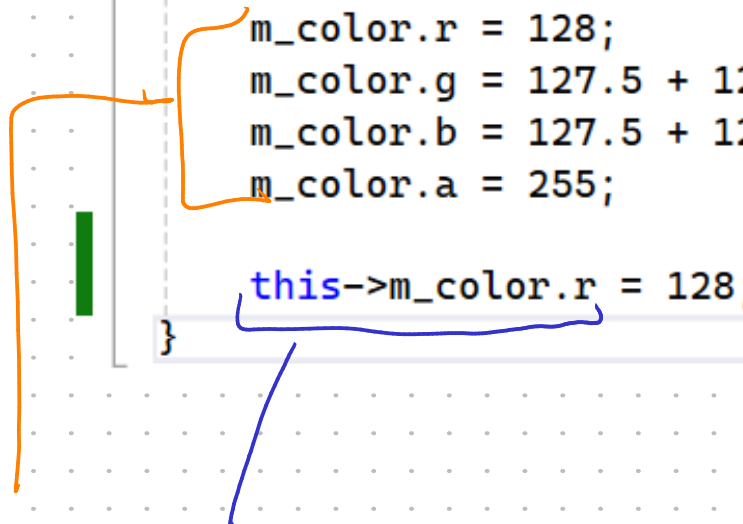
Methodes = fonction membre

Attributes = variable membre

Attributes = variable membre

```
void exo2::Pong::Update(double absTime, float deltaTime)
{
    // UPDATE
    m_color.r = 128;
    m_color.g = 127.5 + 127.5 * std::sin(absTime * 3);
    m_color.b = 127.5 + 127.5 * std::cos(absTime * 3);
    m_color.a = 255;

    this->m_color.r = 128;
}
```

An orange bracket on the left side of the code block groups the four lines of the 'UPDATE' section (m_color.r, m_color.g, m_color.b, m_color.a). A vertical green bar is positioned to the left of this bracket. A blue bracket underlines the line 'this->m_color.r = 128;'. A blue arrow points from the handwritten text 'Accès EXPLICITE' to this line. An orange arrow points from the handwritten text 'Accès IMPLICITE' to the orange bracket.

Accès IMPLICITE aux variables membres

Accès EXPLICITE aux variables membres
par le pointeur this passé implicitement
par le compilateur

```
31  
32 void exo2::Pong::Draw() const  
33 {  
34     // DRAW  
35     m_color.r = 128;  
36     jv::gpu::SetBackgroundColor(m_color);  
37 }  
38
```

146 %

Error List

Entire Solution 1 Error 0 Warnings 0 Messages Build Only

	Code	Description
✖	C2166	l-value specifies const object

Left - value = Valeur à gauche de l'assignement

```

32  void exo2::Pong::Draw() const
33  {
34      // DRAW
35      jv::gpu::SetBackgroundColor(m_color);
36
37      exo2::DrawRect({}, {10, 10}, {});
38  }
39

```

Ln: 37 Ch: 38 SPC				
ist				
Solution 2 Errors 0 Warnings 0 Messages Build Only				
Search Error List				
Code	Description	Project	File	Line
LNK2019	unresolved external symbol "void __cdecl exo2::DrawRect(struct jv::util::Vec2,struct jv::util::Vec2,union jv::util::Color)" (?DrawRect@exo2@@@YAXUVec2@util@jv@@@0TColor@34@@@Z) referenced in function "public: void __cdecl exo2::Pong::Draw(void)const" (?Draw@Pong@exo2@@@QEBAXXZ)	Exo2	exo2_pong.obj	1
LNK1120	1 unresolved externals	Exo2	Exo2.exe	1

"J'ai pas trouvé"

LNK → Link → Lien

→ Manque une définition de DrawRect

exo1_pong.cpp x exo2_draw.cpp exo2_draw.h exo2_pong.h exo2_pong.cpp exo2_main.cpp exo2.md

Exo1 (Global Scope)

```
22 constexpr Color kDelimColor = {100, 150, 100, 255};
```

```
23
```

```
24 // Constantes des dimensions des objets.
```

```
25 constexpr Vec2 kArenaSize = {300, 200};
```

```
26 constexpr Vec2 kBallSize = {10, 10};
```

```
27 constexpr Vec2 kPlayerSize = {10, 40};
```

```
28 constexpr Vec2 kAiSize = {10, 100};
```

```
29
```

```
30 // Variables globales
```

```
31 Vec2 g_playerPos;
```

```
32 Vec2 g_aiPos;
```

```
33 Vec2 g_ballPos;
```

```
34 Vec2 g_ballDir;
```

```
35 float g_ballSpeed;
```

```
36
```

```
37 > void DrawRect(Vec2 pos, Vec2 size, Color color) { ... }
```

```
58
```

```
59 > void jv::game::Init() { ... }
```

```
75
```

```
76 > void jv::game::Update(double absTime, float deltaTime) { ... }
```

```
148
```

```
149 > void jv::game::Draw() { ... }
```

```
160
```

```
161 v void jv::game::Shut()
```

```
162 {
```

```
163 }
```

```
164
```

CONSTANTES

VAR GLOBALES

FONCTION UTILITAIRE

FONCTIONS LIBRES
POUR LA LOGIQUE
DE JEU

```

exo1_pong.cpp x exo2_draw.cpp exo2_draw.h exo2_pong.h exo2_pong.cpp exo2_main.cpp exo2.md
Exo1 (Global Scope)
22 constexpr Color kDelimColor = {100, 150, 100, 255};
23
24 // Constantes des dimensions des objets.
25 constexpr Vec2 kArenaSize = {300, 200};
26 constexpr Vec2 kBallSize = {10, 10};
27 constexpr Vec2 kPlayerSize = {10, 40};
28 constexpr Vec2 kAiSize = {10, 100};
29
30 // Variables globales
31 Vec2 g_playerPos;
32 Vec2 g_aiPos;
33 Vec2 g_ballPos;
34 Vec2 g_ballDir;
35 float g_ballSpeed;
36
37 > void DrawRect(Vec2 pos, Vec2 size, Color color) { ... }
58
59 > void jv::game::Init() { ... }
75
76 > void jv::game::Update(double absTime, float deltaTime) { ... }
148
149 > void jv::game::Draw() { ... }
160
161 v void jv::game::Shut()
162 {
163 }
164

```

CONSTANTES → exo2_pong.cpp

VAR GLOBALES → VAR MEMBRES

exo2_draw.cpp

FONCTION UTILITAIRE ↗

FONCTIONS LIBRES
POUR LA LOGIQUE
DE JEU

↘ DANS LES
FONCTIONS
MEMBRES

+ Ctrl+H : Remplacer g- par m-
car var globales deviennent var membres

```

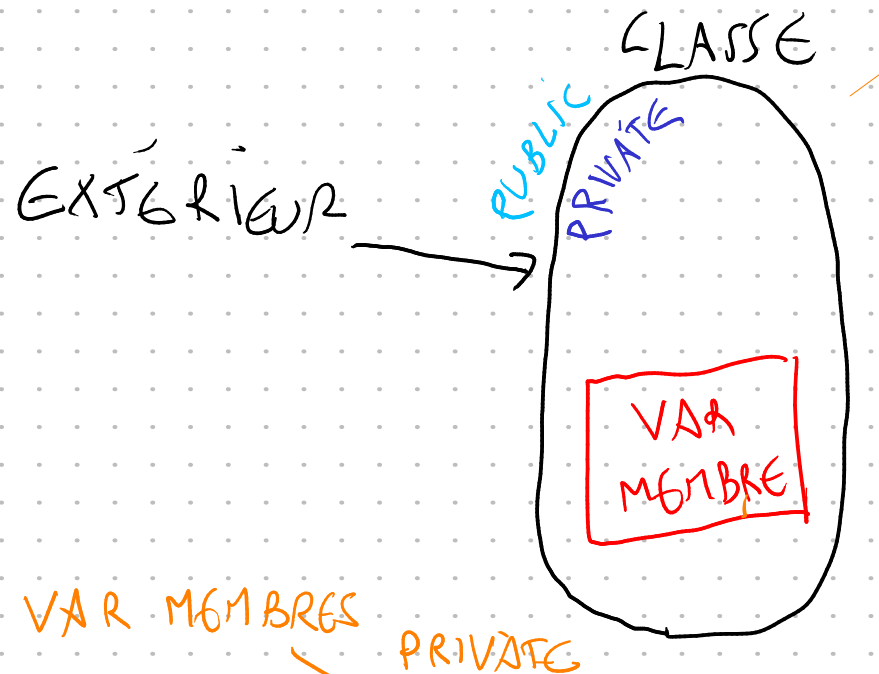
8  class Pong
9  {
10 public:
11     Pong();
12
13     void Update(double absTime, float deltaTime);
14
15     void Draw() const;
16
17 private:
18     using Vec2 = jv::util::Vec2;
19     using Color = jv::util::Color;
20
21     // VARIABLES
22     Vec2 m_playerPos;
23     Vec2 m_aiPos;
24     Vec2 m_ballPos;
25     Vec2 m_ballDir;
26     float m_ballSpeed;
27
28     void _UpdateAI(double absTime, float deltaTime);
29     void _UpdatePlayer(double absTime, float deltaTime);
30     void _UpdateBall(double absTime, float deltaTime);
31 };

```

Privé s'accède que depuis les méthodes de la classe

Intérêt : s'il y a un bug sur le contenu d'une variable, on a juste besoin d'inspecter les méthodes de la classe et non pas toute la base de code.

ENCAPSULATION



= Cache l'état interne d'une classe
pour n'exposer que son comportement
MÉTHODES PUBLIQUES

INVARIANT

→ Condition toujours vraie sur l'état interne de la classe

Ex : - BallSpeed est toujours > 0

- Player / IA / Ball tjs sur le terrain

1. Quand on lit l'état interne, on a une idée plus précise de ce qu'on attend.

2. Si jamais l'invariant est faux, c'est qu'il y a un bug dans les méthodes qui modifient l'état.

Intérêt du constructeur : il est toujours appelé, il est responsable d'initialiser l'état de façon à respecter les invariants.

$center.x = (pointSize.x * pts)$
 $center.y = pointSize.y / 2$

